# Bi-Directional Domain Adaptation for Sim2Real Transfer of Embodied Navigation Agents

Joanne Truong, Sonia Chernova, and Dhruv Batra

***Abstract*—Deep reinforcement learning models are notoriously data hungry, yet real-world data is expensive and time consuming to obtain. The solution that many have turned to is to use simulation for training before deploying the robot in a real environment. Simulation offers the ability to train large numbers of robots in parallel, and offers an abundance of data. However, no simulation is perfect, and robots trained solely in simulation fail to generalize to the real-world, resulting in a "sim-vs-real gap". How can we overcome the trade-off between the abundance of less accurate, artificial data from simulators and the scarcity of reliable, real-world data? In this letter, we propose Bi-directional Domain Adaptation (BDA), a novel approach to bridge the sim-vs-real gap in both directions–*real2sim* to bridge the visual domain gap, and *sim2real* to bridge the dynamics domain gap. We demonstrate the benefits of BDA on the task of PointGoal Navigation. BDA with only 5 k real-world (state, action, next-state) samples matches the performance of a policy fine-tuned with ∼600 k samples, resulting in a speed-up of $\sim 120\times$.**

***Index Terms*—AI-Enabled Robotics, reinforcement learning, vision-based navigation.**

## I. Introduction

DEEP reinforcement learning (RL) methods have made tremendous progress in many high-dimensional tasks, such as navigation [1], manipulation [2], and locomotion [3]. Since RL algorithms are data hungry, and training robots in the real-world is slow, expensive, and difficult to reproduce, these methods are typically trained in simulation (where gathering experience is scalable, safe, cheap, and reproducible) before being deployed in the real-world.

However, no simulator perfectly replicates reality. Simulators fail to model many aspects of the robot and the environment (noisy dynamics, sensor noise, wear and-tear, battery drainage, e.t.c.). In addition, RL algorithms are prone to overfitting – i.e., they learn to achieve strong performance in the environments they were trained in, but fail to generalize to novel environments. On the other hand, humans are able to quickly adapt to small changes in their environment. The ability to quickly adapt and transfer skills is a key aspect of intelligence that we hope to reproduce in artificial agents.

This raises a fundamental question – How can we leverage imperfect but useful simulators to train robots while ensuring that the learned skills generalize to reality? This question is studied under the umbrella of 'sim2real transfer' and has been a topic of much interest in the community [4]–[10].

In this work, we first reframe the sim2real transfer problem into the following question – *given a cheap abundant low-fidelity data generator (a simulator) and an expensive scarce high-fidelity data source (reality), how should we best leverage the two to maximize performance of an agent in the expensive domain (reality)?* The status quo in machine learning is to pre-train a policy in simulation using large amounts of simulation data (potentially with domain randomization [8]) and then fine-tune this policy on the robot using the small amount of real data. Can we do better?

We contend that the small amount of expensive, high-fidelity data from reality is better utilized to adapt the simulator (and reduce the sim-vs-real gap) than to directly adapt the policy. Concretely, we propose Bi-directional Domain Adaptation (BDA) between simulation and reality to answer this question. BDA reduces the sim-vs-real gap in two different directions (shown in Fig. 1).

First, for sensory observations (e.g. an RGB-D camera image $I$) we train a *real2sim* observation adaptation module $\mathcal{OA}: \mathcal{I}^{\text{real}} \mapsto \mathcal{I}^{\text{sim}}$. This can be thought of as 'goggles' [10], [11] that the agent puts on at deployment time to make real observations 'look' like the ones seen during training in simulation. At first glance, this choice may appear counter-intuitive (or the 'wrong' direction). We choose real2sim observation adaption instead of sim2real because this decouples sensing and acting. If the sensor characteristics in reality change but the dynamics remain the same (e.g. same robot different camera), the policy does not need to be retrained, but only equipped with a re-trained observation adaptor. In contrast, changing a sim2real observation adaptor results in the generated observations being out of distribution for the policy, requiring expensive re-training of the policy. Our real2sim observation adaptor is based on CycleGAN [12], and thus does *not* require any sort of alignment or pairing between sim and real observations, which can be prohibitive.

Second, for transition dynamics $\mathcal{T}: Pr(s_{t+1} \mid s_t, a_t)$ (the probably of transitioning from state $s_t$ to $s_{t+1}$ upon taking action $a_t$), we train a *sim2real* dynamics adaptation module $\mathcal{DA}: \mathcal{T}^{\text{sim}} \mapsto \mathcal{T}^{\text{real}}$. This can be thought of as a neural-augmented

Manuscript received October 15, 2020; accepted February 16, 2021. Date of publication February 25, 2021; date of current version March 17, 2021. This letter was recommended for publication by Associate Editor E. Ricci and Editor E. Marchand upon evaluation of the reviewers' comments. The Georgia Tech effort was supported by NSF, AFRL, DARPA, ONR YIPs, ARO PECASE. The work of Joanne Truong was supported by an NSF Graduate Research Fellowship under Grant DGE-1650044 and a Google Women Techmaker's Fellowship. *(Corresponding author: Joanne Truong.)*

Joanne Truong is with the Georgia Institute of Technology, Atlanta, GA 30332 USA (e-mail: truong.j@gatech.edu).

Sonia Chernova and Dhruv Batra are with the Georgia Institute of Technology, Atlanta, GA 30332 USA, and also with the Facebook AI Research, Menlo Park 94025 USA (e-mail: chernova@cc.gatech.edu; dbatra@gatech.edu).

This letter has supplementary downloadable material available at https://doi.org/10.1109/LRA.2021.3062303, provided by the authors.

Digital Object Identifier 10.1109/LRA.2021.3062303

2377-3766 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:55:33 UTC from IEEE Xplore. Restrictions apply.

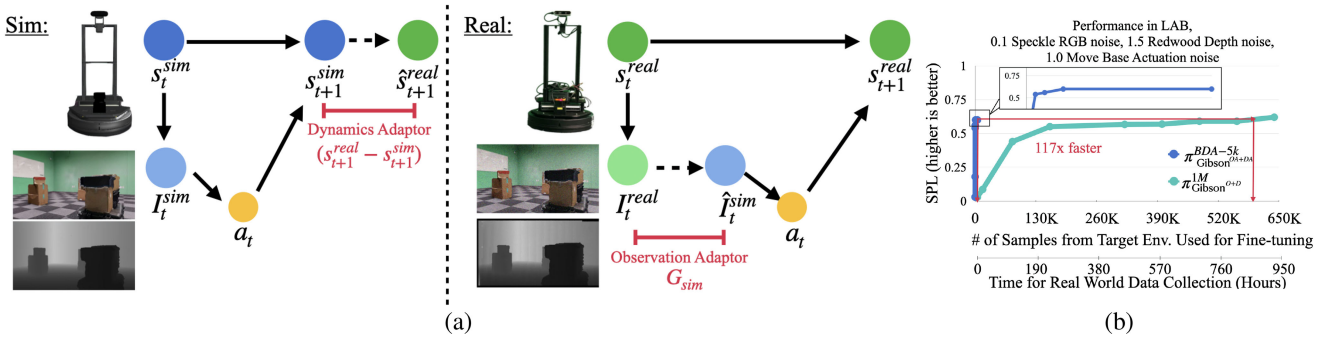



Fig. 1. (a) Left: We learn a *sim2real* dynamics adaptation module to predict residual errors between state transitions in simulation and reality. Right: We learn a *real2sim* observation adaptation module to translate images the robot sees in the real-world at test time to images that more closely align with what the robot has seen in simulation during training. (b) Using BDA, we achieve the same SPL as a policy finetuned directly in reality while using 117× less real-world data.

simulator [5] or a specific kind of boosted ensembling method [13] – where a simulator first makes predictions about state transitions and then a learned neural network predicts the residual between the simulator predictions and the state transitions observed in reality. At each time $t$ during training in simulation, $\mathcal{DA}$ resets the simulator state from $s^{\text{sim}}_{t+1}$ (where the simulator believes the agent should reach at time $t+1$) to $\hat{s}^{\text{real}}_{t+1}$ (where $\mathcal{DA}$ predicts the agent will reach in reality), thus exposing the policy to trajectories expected in reality. We choose sim2real dynamics adaptation instead of real2sim because this nicely exploits the fundamental asymmetry between the two domains – simulators can (typically) be reset to arbitrary states, reality (typically) cannot. Once an agent acts in the real-world, it doesn't matter what corresponding state it would have reached in simulator, reality cannot be reset to it.

Once the two modules are trained, BDA trains a policy in a simulator augmented with the dynamics adaptor ($\mathcal{DA}$) and deploys the policy augmented with the observation adaptor ($\mathcal{OA}$) to reality. This process is illustrated in Fig. 1(a), left showing policy training in simulation and right showing its deployment in reality.

We instantiate and demonstrate the benefits of BDA on the task of PointGoal Navigation (PointNav) [14], which involves an agent navigating in a previously unseen environment from a randomized initial starting location to a goal location specified in relative coordinates. For controlled experimentation, and due to COVID-19 restrictions, we use Sim2Sim transfer of PointNav policies as a stand-in for Sim2Real transfer. We conduct experiments in photo-realistic 3D simulation environments using Habitat-Sim [15], which prior work [16] has found to have high *sim2real predictivity*, meaning that inferences drawn in simulation experiments have a high likelihood of holding in reality on Locobot mobile robot [17].

In our experiments, we find that BDA is *significantly* more sample-efficient than the baseline of fine-tuning a policy. Specifically, BDA trained on as few as 5000 samples (state, action, next-state) from reality (equivalent of 7 hours to collect data in reality) is able to match the performance of baseline trained on *585000* samples from reality (equivalent of 836 hours to collect data in reality, or 3.5 months at 8 working hours per day), a speed-up of 117× (Fig. 1(b)).

**Algorithm 1:** Bi-Directional Domain Adaptation.

Train behavior policy $\pi_{\text{sim}}$ in Sim
**for** $t = 0,...,$ N steps **do**
  Collect $\mathcal{I}^{\text{sim}}_t \sim$ Sim rollout ($\pi_{\text{sim}}$)
  Collect $\mathcal{I}^{\text{real}}_t$, $s^{\text{real}}_t$, $a^{\text{real}}_t \sim$ Real rollout ($\pi_{\text{sim}}$)
Train $\mathcal{OA}$ ($\{I^{\text{sim}\,N}_{i=1}\}$, $\{I^{\text{real}\,N}_{i=1}\}$)
Train $\mathcal{DA}$ ($\{s^{\text{real}\,N}_{i=1}\}$, $\{a^{\text{real}\,N}_{i=1}\}$)
$\text{Sim}^{\mathcal{DA}} \leftarrow$ Augment Source with $\mathcal{DA}$
**for** $j = 0,...,$ K steps **do**
  $\pi_{\text{Sim}^{\mathcal{DA}}} \leftarrow$ Finetune $\pi_{\text{sim}}$ in $\text{Sim}^{\mathcal{DA}}$
$\pi_{\text{Sim}^{\mathcal{OA}+\mathcal{DA}}} \leftarrow$ Apply $\mathcal{OA}$ at test-time
Test $\pi_{\text{Sim}^{\mathcal{OA}+\mathcal{DA}}}$ in Real

While our experiments are conducted on the PointNav task, we believe our findings, and the core idea of Bi-directional Domain Adaptation, is broadly applicable to a number of problems in robotics and reinforcement learning.

## II. Bi-directional Domain Adaptation (BDA)

We now describe the two key components of Bi-directional Domain Adaptation (BDA) in detail – (1) *real2sim* observation adaptation module $\mathcal{OA}$ to close the visual domain gap, and (2) *sim2real* dynamics adaptation module $\mathcal{DA}$ to close the dynamics domain gap.

*Preliminaries and Notation:* We formulate our problem by representing both the source and target domain as a Markov Decision Process (MDP). A MDP is defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma)$, where $s \in \mathcal{S}$ denotes states, $a \in \mathcal{A}$ denotes actions, $\mathcal{T}(s, a, s') = Pr(s' \mid s, a)$ is the transition probability, $\mathcal{R} : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is the reward function, and $\gamma$ is a discount factor. In RL, the goal is to learn a policy $\pi : \mathcal{S} \to \mathcal{A}$ to maximize expected reward.

### A. System Architecture

*Observation Adaptation:* We consider a *real2sim* domain adaptation approach to deal with the visual domain gap.

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:55:33 UTC from IEEE Xplore. Restrictions apply.

We leverage CycleGAN [12], a pixel-level image-to-image translation technique that uses a cycle-consistency loss function with unpaired images. We start by using a behavior policy $\pi_{\text{sim}}$ trained in simulation to sample rollouts in simulation and reality to collect RGB-D images $\mathcal{I}_t^{\text{sim}}$ and $\mathcal{I}_t^{\text{real}}$ at time $t$ (line 3). The dataset of N unpaired images $\{I^{\text{sim}}{}_{i=1}^{N}\}$ and $\{I^{\text{real}}{}_{i=1}^{N}\}$ is used to train $\mathcal{OA}$, to translate $\{I^{\text{sim}}{}_{i=1}^{N}\} \mapsto \{I^{\text{real}}{}_{i=1}^{N}\}$ (line 5). $\mathcal{OA}$ learns a mapping $\mathcal{G}_{\text{sim}} : \mathcal{I}^{\text{real}} \mapsto \mathcal{I}^{\text{sim}}$, an inverse mapping $\mathcal{G}_{\text{real}} : \mathcal{I}^{\text{sim}} \mapsto \mathcal{I}^{\text{real}}$, and adversarial discriminators $\mathcal{D}_{\text{real}}$, $\mathcal{D}_{\text{sim}}$. Although our method focuses on adaptation from *real2sim*, learning both mappings encourages the generative models to remain cycle-consistent, i.e., forward cycle: $\mathcal{I}^{\text{real}} \rightarrow \mathcal{G}_{sim}(\mathcal{I}^{\text{real}}) \rightarrow \mathcal{G}_{\text{real}}(\mathcal{G}_{\text{sim}}(\mathcal{I}^{\text{real}})) \approx \mathcal{I}^{\text{real}}$ and backwards cycle: $\mathcal{I}^{\text{sim}} \rightarrow \mathcal{G}_{\text{real}}(\mathcal{I}^{\text{sim}}) \rightarrow \mathcal{G}_{\text{sim}}(\mathcal{G}_{\text{real}}(\mathcal{I}^{\text{sim}})) \approx \mathcal{I}^{\text{sim}}$. The ability to learn mappings from unpaired images from both domains is important because it is difficult to accurately collect paired images between simulation and reality.

A *real2sim* approach for adapting the visual domain offers many advantages over a *sim2real* approach because it disentangles the sensor adaptation module from our policy training. This enables us to remove an additional bottleneck during the RL policy training process; we can train $\mathcal{OA}$ in parallel with the RL policy, thus reducing the overall training time needed. In addition, if the sensor observation noise in the environment changes, the base policy can be kept frozen, and only $\mathcal{OA}$ will have to be retrained.

*Dynamics Adaptation:* To close the dynamics domain gap, we follow a *sim2real* approach. Starting with the behavior policy $\pi_{\text{sim}}$, we collect state-action pairs ($s_t^{\text{real}}$, $a_t^{\text{real}}$) in the real-world (line 4). The state-action pairs are used to train $\mathcal{DA}$, a 3 layer multilayer perceptron regression network, that learns the residual error between the state transitions in simulation and reality $\mathcal{T}^{\text{sim}} \mapsto \mathcal{T}^{\text{real}}$ (line 7). Specifically, $\mathcal{DA}$ learns to estimate the change in position and orientation $\Delta s^{\text{real}} : (s_{t+1}^{\text{real}} - s_t^{\text{real}})$. We use a weighted MSE loss function, $\frac{1}{n}\sum_{n=1}^{N} \mathbf{w}^{\top}(\Delta s_n^{\text{real}} - \Delta \hat{s}_n^{\text{real}})^2$. For our experiments, the state $s_t^{\text{real}} = (x_t^{\text{real}}, y_t^{\text{real}}, \theta_t^{\text{real}})$, is represented by the position and orientation of the robot at timestep $t$. We placed twice as much weight on the prediction terms for the robot's position than for its orientation because getting the position correct is more important for our performance metric. Once trained, $\mathcal{DA}$ is used to augment the source environment (line 7). We finetune $\pi_{\text{sim}}$ in the augmented simulator, $\text{Sim}^{\mathcal{DA}}$ (lines 8-9). Our hypothesis (which we validate in our experiments) is using real-world data to adapt the simulator via our $\mathcal{DA}$ model pays off because we can then train RL policies in this $\mathcal{DA}$-augmented simulator for large amounts of experience cheaply. We use $\mathcal{OA}$ at test time (line 10). Finally, we test our policy trained with BDA in the real-world (line 11).

To recap, BDA has a number of advantages over the status quo (of directly using real data to fine-tune a simulation trained policy) that we demonstrate in our experiments:

1) Decouples sensing and acting,
2) Does not require paired training data,
3) The data to train both modules can be collected jointly (by gathering experience from a behavior policy in reality) but the two can be trained in parallel independently of each other,
4) Similar to model-based RL [18], reducing the sim-vs-real gap is made significantly more sample-efficient than directly fine-tuning the policy.

## III. Experimental Setup: Sim2Sim Transfer for Point-Goal Navigation

Our goal in this work is to enable sample efficient Sim2Real transfer for the task of PointGoal Navigation (PointNav) [14]. However, for controlled experiments and due to COVID-19 restrictions, we study Sim2Sim transfer as a stand-in for Sim2Real. Specifically, we train policies in a "source" simulator (which serves as 'Sim' in 'Sim2Real') and transfer it to a "target" simulator (which serves as 'Real' in 'Sim2Real'). We add observation and dynamics noise to the target simulator to mimic the noise observed in reality. Notice that these noise models are purely for the purpose of conducting controlled experiments and are not available to the agent (which must adapt and learn from samples of state and observations). Since no noise model is perfect (just like no simulator is perfect), we experiment with a range of noise models and report results with multiple target simulators. Our results show consistent improvements regardless of the noise model used, thus providing increased confidence in our experimental setup. For clarity, in the text below we present our approach from the perspective of "transfer from a source to target domain," with the assumption that obtaining data in the target domain is always expensive, regardless of whether it is a simulated or real-world environment. All of our experiments are conducted in Habitat [15].

### A. Task: PointGoal Navigation

In PointNav, a robot is initialized in an unseen environment and asked to navigate to a goal location specified in relative coordinates purely from egocentric RGB-D observations without a map, in a limited time budget. An episode is considered successful if the robot issues the `STOP` command within 0.2 m of the goal location. In order to increase confidence that our simulation settings will translate to the real-world, we limit episodes to 200 steps, limit number of collisions allowed (before deeming the episode a failure) to 40, and turn sliding off– specifications found by [16] to have high sim2real predictivity (how well evaluation in simulation predicts real-world performance). Sliding is a behavior enabled by default in many physics simulators that allows agents to slide along obstacles when the agent takes an action that would result in a collision. Turning sliding off ensures that the agent cannot cheat in simulation by sliding along obstacles. We use success rate (SUCC), and Success weighted by (normalized inverse) Path Length (SPL) [14] as metrics for evaluation.

### B. Robot in Simulation

*Body:* The robot has a circular base with a radius of 0.175 m and a height of 0.61 m. These dimensions correspond to the base width and camera height of the LoCoBot robot [17].

*Sensors:* The robot has access to an egocentric RGB and Depth sensor, and accurate localization and heading through a GPS+Compass sensor. Real-world robot experiments from [16]

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:55:33 UTC from IEEE Xplore. Restrictions apply.

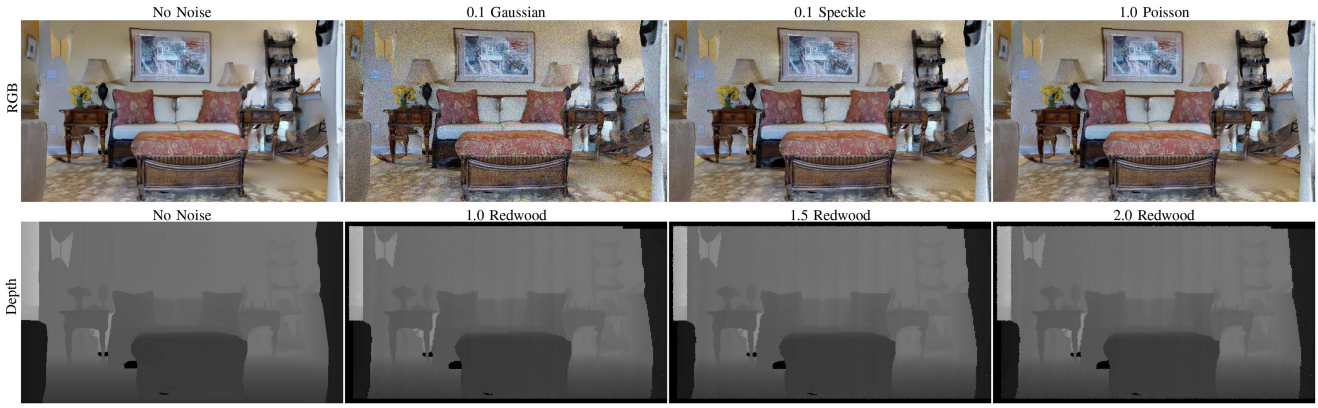



Fig. 2. Different RGB and Depth sensor noise combinations we apply to our training and testing environments. From left to right: 0 RGB noise + 0 Depth noise, 0.1 Gaussian RGB noise + 1.0 Redwood Depth noise, 0.1 Speckle RGB noise + 1.5 Redwood Depth noise, 1.0 Poisson RGB noise + 2.0 Redwood Depth noise.

used Hector SLAM [19] with a Hokuyo UTM-30LX LIDAR sensor and found that localization errors were approximately 7 cm (much lower than the 20 cm PointNav success criterion). This gives us confidence that our results will generalize to reality, despite the lack of precise localization. We match the specifications of the Intel D435 camera on the LoCoBot, and set the camera field of view to 70. To match the maximum range on the depth camera, we clip the simulated depth readings to 10 m.

*Sensor Noise:* To simulate noisy sensor observations of the real-world, we add RGB and Depth sensor noise models to the simulator. Specifically, we use Gaussian, Speckle, and Poisson noise for the RGB camera, and Redwood noise for the Depth camera. More details about the Redwood noise can be found in [20]. Fig. 2 shows a comparison between noise free RGB-D images and RGB-D images with the different noise models and multipliers we use.

*Actions:* The action space for the robot is `turn-left 30°` `turn-right 30°`, `forward 0.25 m`, and `STOP`. In the source simulator, these actions are executed deterministically and accurately. However, actions in the real-world are never deterministic – identical actions can lead to vastly different final locations due to the actuation noise (wheel slippage, battery power drainage, etc.) typically found on a real robot. To simulate the noisy actuation that occurs in the real-world, we leverage the real-world translational and rotational actuation noise models characterized by [21]. A Vicon motion capture was used to measure the difference in commanded state and achieved state on LoCoBot for 3 different positional controllers: Proportional Controller, Dynamic Window Approach Controller from Movebase, and Linear Quadratic Regulator (ILQR). These are controllers typically used on a mobile robot. From a state (x, y, $\theta$) and given a particular action, we add translational noise sampled from a truncated 2D Gaussian, and rotational noise from a 1D Gaussian to calculate the next state.

### C. Testing Environment

We virtualize a 6.5 m by 10 m real lab environment (LAB) to use as our testing environment, using a Matterport Pro2 3D camera. To model the space, we placed the Matterport camera at various locations in the room, and collected 360° scans of the environment. We used the scans to create 3D meshes of the environment, and directly imported the 3D meshes into Habitat




(a) (b)

Fig. 3. (a) Top-down view of one of our testing environments. White boxes are obstacles. The robot navigates sequentially through the waypoints $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow A$. Figure taken from [16]. (b) 3D visualization of the robot navigating in one of our testing environments in simulation. RGB and Depth observations are shown on the right.

to create a photorealistic replica of LAB Fig. 3(b). We vary the number of obstacles in LAB to create 3 room configurations with varying levels of difficulty. Fig. 3 shows one of our room configurations with 5 obstacles. We perform testing over the 3 different room configurations, each with 5 start and end waypoints for navigation episodes, and 10 independent trials, for a total of 150 runs. We report the average success rate and SPL over the 150 runs.

Our models were trained entirely in the Gibson dataset [11], and have never seen LAB during training. The Gibson dataset contains 3D models of 572 cluttered indoor environments (homes, hospitals, offices, museums, etc.). In this work, we used the 72 Gibson environments that were rated 4+ in quality in [15].

### D. Experimental Protocol

Recall that our objective is to improve the ability for RL agents to generalize to new environments using little real-world data. To do this, we define our source environment as Gibson without any sensor or actuation noise (Gibson$^{\text{no_noise}}$). We create 10 target environments with noise settings described in Table I. We use the notation $O$ to represent an environment afflicted with only RGBD observation noise (rows 2, 5, or 8), $D$ to represent an environment afflicted with only dynamics noise (rows 3, 6, or 9), and $O + D$ to represent an environment afflicted with RGBD observation noise and dynamics noise (rows 4, 7, or 10).

### E. RL Navigation Models

We train learning-based navigation policies, $\pi$, for PointNav in Habitat using environments from the Gibson dataset. Policies

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:55:33 UTC from IEEE Xplore. Restrictions apply.

TABLE I
DEFINITION OF THE 10 DIFFERENT NOISE SETTINGS WE USE FOR TRAINING AND TESTING. ROW 1 INDICATES THE 'SOURCE' ENVIRONMENT WITH NO OBSERVATION OR ACTUATION NOISE PRESENT

| # | RGB Obs Noise | Depth Obs Noise | Actuation Noise |
|---|---|---|---|
| 1 | - | - | - |
| 2 | Gaussian 0.1 | Redwood 1.0 | |
| 3 | - | - | Proportional 1.0 |
| 4 | Gaussian 0.1 | Redwood 1.0 | Proportional 1.0 |
| 5 | Speckle 0.1 | Redwood 1.5 | |
| 6 | - | - | Move Base 1.0 |
| 7 | Speckle 0.1 | Redwood 1.5 | Move Base 1.0 |
| 8 | Poisson 1.0 | Redwood 2.0 | - |
| 9 | - | - | ILQR 1.0 |
| 10 | Poisson 1.0 | Redwood 2.0 | ILQR 1.0 |

were trained from scratch with reinforcement learning using DD-PPO [1], a decentralized, distributed variant of the proximal policy optimization (PPO) algorithm, that allows for large-scale training in GPU-intensive simulation environments. We follow the navigation policy described in [1], which is composed of a ResNet50 visual encoder, and a 2-layer LSTM. Each policy is trained using 64 Tesla V100 s. Base policies are trained for 100 million steps ($\pi^{100\,\mathrm{M}}$) to ensure convergence.

## IV. EXPERIMENTS

Our experiments aim to answer the following: (1) How large is the sim2real gap? (2) Does our method improve generalization to target domains? (3) How does our method compare to directly training (or fine-tuning) in the target environment? (4) How much real-world data do we need?

*How large is the sim2real gap?* First, we show that RL policies fail to generalize to new environments. We train a policy without any noise ($\pi^{100\,\mathrm{M}}_{\mathrm{Gibson}^{\mathrm{no_noise}}}$), and a policy with observation and dynamics noise ($\pi^{100\,\mathrm{M}}_{\mathrm{Gibson}^{O+D}}$). We test these policies in LAB with 4 different noise settings: $\mathrm{LAB}^{\mathrm{no_noise}}$, $\mathrm{LAB}^{O}$, $\mathrm{LAB}^{D}$, $\mathrm{LAB}^{O+D}$, and average across the noise settings. For each noise setting, we conduct 3 sets of runs, each containing 150 episodes in the target environments. We see that $\pi^{100\,\mathrm{M}}_{\mathrm{Gibson}^{\mathrm{no_noise}}}$ tested in $\mathrm{LAB}^{\mathrm{no_noise}}$ exhibits good transfer across environments – 0.84 SPL (in contrast, the Habitat 2019 challenge winner was at 0.95 SPL [22]). [1] showed that near-perfect performance is possible when the policy is trained out for significantly longer (2.5B frames), but for the sake of multiple experiments, we limit our analysis to 100 M frames of training and compare all models across the same number.

From Fig. 4, we see that when dynamics noise is introduced ($\pi^{100\,\mathrm{M}}_{\mathrm{Gibson}^{\mathrm{no_noise}}}$ tested in $\mathrm{LAB}^{D}$), SPL drops from 0.84 to 0.56 (relative drop of 28%). More significantly, when sensor noise is introduced ($\pi^{100\,\mathrm{M}}_{\mathrm{Gibson}^{\mathrm{no_noise}}}$ tested in $\mathrm{LAB}^{O}$), SPL drops to 0.04 (relative drop of 81%), and when both sensor and dynamics noise are present, ($\pi^{100\,\mathrm{M}}_{\mathrm{Gibson}^{\mathrm{no_noise}}}$ tested in $\mathrm{LAB}^{O+D}$), SPL drops to 0.06 (relative drop of 78%). Thus, in the absence of noise, generalization across scenes (Gibson to LAB) is good, but in the presence of noise, the generalization suffers. We also notice that the converse is true: policies trained from scratch in $\mathrm{Gibson}^{O+D}$ environments fail to generalize to $\mathrm{LAB}^{\mathrm{no_noise}}$ and $\mathrm{LAB}^{D}$ environments. These results show us that RL agents are highly sensitive to what might be considered perceptually minor changes to visual inputs. To the best of our knowledge, no prior work in embodied navigation appears to have considered this question of sensitivity to noise; hopefully our results will encourage others to consider this as well.

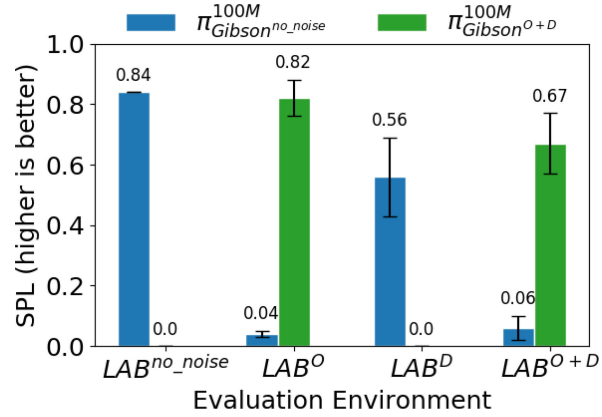



Fig. 4. Zero-shot transfer of $\pi^{100\,\mathrm{M}}_{\mathrm{Gibson}^{\mathrm{no_noise}}}$ and $\pi^{100\,\mathrm{M}}_{\mathrm{Gibson}^{O+D}}$ tested in LAB with different combinations of observation and dynamics noise. We see that SPL drops when a policy is tested in an environment with noise different from what it was trained in.

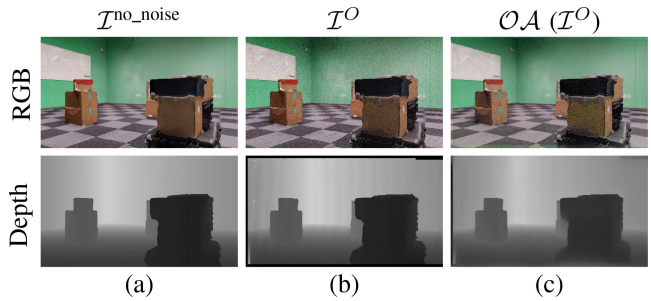



Fig. 5. (a) LAB with no sensor noise (b) LAB with 0.1 Gaussian RGB noise and 1.0 Redwood Depth noise. (c) By adapting images from *real2sim*, we now have images that closely resemble (a).

*How well does $\mathcal{OA}$ do?* Following Alg. 1 described in Sec. II-A, we train $\mathcal{OA}$ from scratch for 200 epochs. In Fig. 5, we see that the model learns to remove the Gaussian noise placed on the RGB image, and learns to smooth out textures in the depth image. In Table II, we see that simply equipping $\pi^{100\,\mathrm{M}}_{\mathrm{Gibson}^{\mathrm{no_noise}}}$ with $\mathcal{OA}$ during deployment ($\pi^{\mathrm{BDA}-5k}_{\mathrm{Gibson}^{OA}}$) drastically improves SPL in $\mathrm{LAB}^{O}$ compared to $\pi^{100\,\mathrm{M}}_{\mathrm{Gibson}^{\mathrm{no_noise}}}$, resulting in an average increase of 65% (rows 2, 6, 10).

In addition, we have RGB-D images of LAB collected from a real robot, pre-COVID, and results using our real2sim $\mathcal{OA}$ module. While no GAN metric is perfect (user studies are typically conducted for evaluation as done in [12]), we calculated the Fréchet Inception Distance (FID) [23] score (lower is better) to provide quantitative results. We find that the FID comparing $I^{\mathrm{real}}$ and $I^{\mathrm{sim}}$ is 100.74, and the FID from $\mathcal{OA}$ ($I^{\mathrm{real}}$) to $I^{\mathrm{sim}}$ images is 83.05. We also calculated FID comparing simulation images afflicted with Gaussian noise, $\mathcal{I}^{\mathrm{Gaussian}}$, to noise-free simulation images $\mathcal{I}^{\mathrm{no_noise}}$ to be 98.73, and FID between $\mathcal{OA}$ ($\mathcal{I}^{\mathrm{Gaussian}}$)

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:55:33 UTC from IEEE Xplore. Restrictions apply.

TABLE II
SUCCESS RATE AND SPL OF FIVE POLICIES WITH RGB-D OBSERVATIONS. $\pi^{100\mathrm{M}}_{\mathrm{Gibson}^{\mathrm{no_noise}}}$ IS A POLICY TRAINED SOLELY IN SIMULATION. $\pi^{\mathrm{BDA-5k}}_{\mathrm{Gibson}^{\mathrm{OA}}}$ IS $\pi^{100\mathrm{M}}_{\mathrm{Gibson}^{\mathrm{no_noise}}}$ EQUIPPED WITH $\mathcal{OA}$ TRAINED USING 5 K IMAGES FROM THE SOURCE AND TARGET ENVIRONMENTS. $\pi^{\mathrm{BDA-5k}}_{\mathrm{Gibson}^{\mathrm{DA}}}$, $\pi^{\mathrm{BDA-5k}}_{\mathrm{Gibson}^{\mathrm{OA+DA}}}$ AND $\pi^{1\mathrm{M}}_{\mathrm{Gibson}^{\mathrm{O+D}}}$ ARE INITIALIZED WITH $\pi^{100\mathrm{M}}_{\mathrm{Gibson}^{\mathrm{no_noise}}}$. $\pi^{\mathrm{BDA-5k}}_{\mathrm{Gibson}^{\mathrm{DA}}}$ IS FINE-TUNED WITH $\mathcal{DA}$ USING 5 K SAMPLES FROM THE TARGET ENVIRONMENT. $\pi^{\mathrm{BDA-5k}}_{\mathrm{Gibson}^{\mathrm{OA+DA}}}$ IS FINETUNED USING THE FULL BDA PIPELINE, UTILIZING BOTH $\mathcal{OA}$ AND $\mathcal{DA}$. $\pi^{1\mathrm{M}}_{\mathrm{Gibson}^{\mathrm{O+D}}}$ IS FINE-TUNED DIRECTLY IN THE TARGET ENVIRONMENT FOR 1 M STEPS OF EXPERIENCE, AND SERVES AS AN ORACLE BASELINE. WHILE $\pi^{1\mathrm{M}}_{\mathrm{Gibson}^{\mathrm{O+D}}}$ AND $\pi^{\mathrm{BDA-5k}}_{\mathrm{Gibson}^{\mathrm{OA+DA}}}$ ACHIEVE IN STRONG PERFORMANCE ACROSS ENVIRONMENTS WITH VARYING NOISES (ROWS 4, 8, 12), BDA REQUIRES 200× FEWER SAMPLES FROM THE TARGET ENVIRONMENT

| # | RGB Obs | Depth Obs | Actuation | $\pi^{100M}_{\mathrm{Gibson}^{\mathrm{no_noise}}}$ | | $\pi^{\mathrm{BDA-5k}}_{\mathrm{Gibson}^{OA}}$ | | $\pi^{\mathrm{BDA-5k}}_{\mathrm{Gibson}^{DA}}$ | | $\pi^{\mathrm{BDA-5k}}_{\mathrm{Gibson}^{OA+DA}}$ | | $\pi^{1M}_{\mathrm{Gibson}^{O+D}}$ | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Noise | Noise | Noise | SUCC | SPL | SUCC | SPL | SUCC | SPL | SUCC | SPL | SUCC | SPL |
| 1 | - | - | - | 1.00 | 0.84 | 1.00 | 0.85 | 1.00 | 0.89 | 0.80 | 0.61 | 0.99 | 0.84 |
| 2 | Gaus. 0.1 | Red. 1.0 | - | 0.10 | 0.04 | 1.00 | 0.78 | 0.21 | 0.10 | 0.97 | 0.80 | 0.99 | 0.87 |
| 3 | - | - | Prop. 1.0 | 0.89 | 0.57 | 0.86 | 0.54 | 1.00 | 0.66 | 0.99 | 0.65 | 1.00 | 0.64 |
| 4 | Gaus. 0.1 | Red. 1.0 | Prop. 1.0 | 0.32 | 0.11 | 0.78 | 0.48 | 0.16 | 0.05 | 1.00 | 0.62 | 1.00 | 0.65 |
| 5 | - | - | - | 1.00 | 0.84 | 1.00 | 0.85 | 0.98 | 0.80 | 0.85 | 0.68 | 0.97 | 0.80 |
| 6 | Speck. 0.1 | Red. 1.5 | - | 0.11 | 0.05 | 1.00 | 0.80 | 0.03 | 0.01 | 0.70 | 0.54 | 0.99 | 0.81 |
| 7 | - | - | MB 1.0 | 0.71 | 0.42 | 0.79 | 0.47 | 1.00 | 0.59 | 0.97 | 0.58 | 1.00 | 0.59 |
| 8 | Speck. 0.1 | Red. 1.5 | MB 1.0 | 0.08 | 0.03 | 0.68 | 0.39 | 0.03 | 0.01 | 0.99 | 0.60 | 1.00 | 0.62 |
| 9 | - | - | - | 1.00 | 0.85 | 1.00 | 0.86 | 1.00 | 0.85 | 1.00 | 0.87 | 1.00 | 0.83 |
| 10 | Pois. 1.0 | Red. 2.0 | - | 0.07 | 0.04 | 0.68 | 0.51 | 0.25 | 0.13 | 0.96 | 0.69 | 1.00 | 0.87 |
| 11 | - | - | ILQR 1.0 | 0.93 | 0.68 | 0.95 | 0.69 | 1.00 | 0.74 | 1.00 | 0.76 | 0.99 | 0.73 |
| 12 | Pois. 1.0 | Red. 2.0 | ILQR 1.0 | 0.14 | 0.05 | 0.63 | 0.39 | 0.25 | 0.08 | 0.99 | 0.63 | 1.00 | 0.73 |

to $\mathcal{I}^{\mathrm{no_noise}}$ images to be 88.44. To put things in context, the FID score comparing images from CIFAR10 to our simulation images is 317.61. This shows that perceptually, the distribution of our adapted images more closely resembles images taken directly from simulation, and that real2sim $\mathcal{OA}$ is not far off from our sim2sim $\mathcal{OA}$ experiments. While our architecture has changed since this initial data collection (initial images are 256 × 256, compared to our current architecture which uses 640 × 360 images), these results will serve as a good indication that our approach will generalize to reality.

How well does $\mathcal{DA}$ do? We train $\mathcal{DA}$ using 5000 samples of state-action pairs collected in the target environment, and augment our source simulator with $\mathcal{DA}$. From Table II, we see that finetuning $\pi^{100\,\mathrm{M}}_{\mathrm{Gibson}^{\mathrm{no_noise}}}$ with $\mathcal{DA}$ ($\pi^{\mathrm{BDA-5}k}_{\mathrm{Gibson}^{DA}}$) on average, leads to a relative 15% improvement in success and a 11% improvement in SPL over $\pi^{100\,\mathrm{M}}_{\mathrm{Gibson}^{\mathrm{no_noise}}}$ in LAB$^D$ (rows 3, 7, 11). Next, we investigate how well our actuation noise model approximates real-world conditions. Using state-action pairs collected from LoCoBot in LAB using the PyRobot proportional controller from our experiments pre-COVID, we trained a $\mathcal{DA}$ module to approximate the translation and rotation noise present in our real-world testing environment. We compare this to the actuation noise models used in our target environments, which were provided from the real-world benchmark by PyRobot [21] using a Vicon motion capture system. Since actuation noise models are a factor of the robot and environment, the $\mathcal{DA}$ learned using our real-world experiments cannot exactly match the noise model benchmarked by PyRobot. However, Table III shows that the noise model learned from LAB is similar in order of magnitude to the noise models derived PyRobot. This gives us confidence that the actuation noise models used in our target simulation as a stand in for reality are a good approximation for the dynamics noise present in the real-world.

How does our method compare to fine-tuning? We evaluate our policy finetuned using BDA with 5000 data samples collected in the target environment ($\pi^{\mathrm{BDA-5}k}_{\mathrm{Gibson}^{OA+DA}}$). We compare this to directly finetuning in the target environment ($\pi^{1\,\mathrm{M}}_{\mathrm{Gibson}^{O+D}}$), which serves as an oracle baseline. Both $\pi^{\mathrm{BDA-5}k}_{\mathrm{Gibson}^{OA+DA}}$ and $\pi^{1\,\mathrm{M}}_{\mathrm{Gibson}^{O+D}}$ are initialized with $\pi^{100\,\mathrm{M}}_{\mathrm{Gibson}^{\mathrm{no_noise}}}$, and both are retrained for each target $O + D$ setting.

TABLE III
AVERAGE TRANSLATION AND ROTATION ACTUATION ERROR FOR LOCOBOT USING THE PYROBOT PROPORTIONAL CONTROLLER. FOR A GIVEN ACTION, THE ACTUATION ERROR IS SAMPLED FROM THE NOISE MODELS, AND ADDED TO THE ACTION TO CALCULATE THE NEXT STATE. WE REPORT THE NOISE MODEL CHARACTERIZED BY REAL-WORLD BENCHMARKING BY PYROBOT, AS WELL AS THE LEARNED $\mathcal{DA}$ NOISE MODEL FROM REAL-WORLD EXPERIMENTS IN LAB

| | X error (mm) | Y error (mm) | $\theta$ error (rad) |
|---|---|---|---|
| **PyRobot** | | | |
| **Linear motion** | 0.042 ±0.15 | 0.017 ±0.08 | 0.031 ±0.16 |
| **Rotation motion** | 0.005 ±0.06 | 0.001 ±0.03 | 0.043 ±0.13 |
| **LAB** | | | |
| **Linear motion** | 0.093 ±0.08 | 0.016 ±0.15 | 0.002 ±0.00 |
| **Rotation motion** | 0.001 ±0.00 | 0.012 ±0.01 | 0.004 ±0.01 |

Our results in Table II show the benefits in finetuning with BDA using data from target environments. While $\pi^{\mathrm{BDA-5}k}_{\mathrm{Gibson}^{OA}}$ and $\pi^{\mathrm{BDA-5}k}_{\mathrm{Gibson}^{DA}}$ show improvements over the baseline policy, both policies still fail to generalize to environments in which new noise is present. Specifically, $\pi^{\mathrm{BDA-5}k}_{\mathrm{Gibson}^{OA}}$ fails to generalize to LAB$^D$ and LAB$^{O+D}$ environments, and $\pi^{\mathrm{BDA-5}k}_{\mathrm{Gibson}^{DA}}$ fails to generalize to LAB$^O$ and LAB$^{O+D}$ environments. On the other hand, both $\pi^{\mathrm{BDA-5}k}_{\mathrm{Gibson}^{OA+DA}}$ and $\pi^{1\,\mathrm{M}}_{\mathrm{Gibson}^{O+D}}$ demonstrate robustness in all combinations of sensor and actuation noise. We also observe that using BDA to learn the observation and dynamics noise models with 5000 samples from the target environment is capable of nearly matching performance of $\pi^{1\,\mathrm{M}}_{\mathrm{Gibson}^{O+D}}$. In fact,

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:55:33 UTC from IEEE Xplore. Restrictions apply.

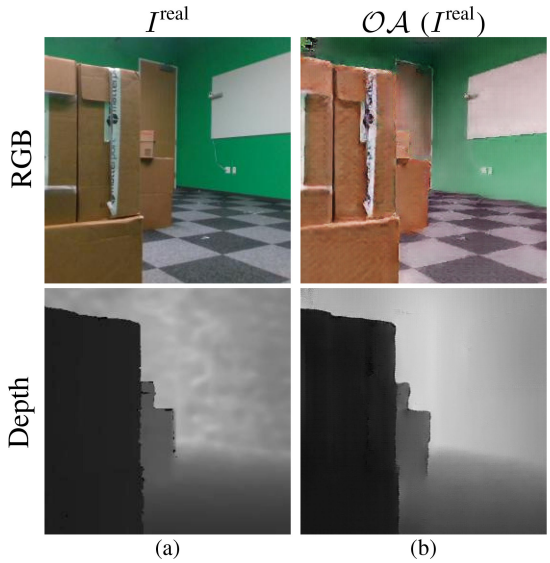



Fig. 6. (a) LAB$^{\text{real}}$ (b) We adapt from *real2sim* to obtain images that closely resemble RGB-D images from simulation.

we only see, on average, a 5% difference between $\pi^{1\text{ M}}_{\text{Gibson}^{O+D}}$ and $\pi^{\text{BDA}-5k}_{\text{Gibson}^{OA+DA}}$ (rows 4, 8, 12), while the former is directly trained in the target environment which is not possible in reality, as it requires 1 M samples from the target environment. In contrast, we see on average, a 25% difference between $\pi^{1\text{ M}}_{\text{Gibson}^{O+D}}$ and $\pi^{\text{BDA}-5k}_{\text{Gibson}^{OA}}$, and an average 62% difference between $\pi^{1\text{ M}}_{\text{Gibson}^{O+D}}$ and $\pi^{\text{BDA}-5k}_{\text{Gibson}^{DA}}$ (rows 4, 8, 12). This highlights the importance of our proposed framework to close the reality gap in both directions; to utilize both real2sim observation adaptation and sim2real dynamics adaptation to accommodate for variations that are overlooked by approaches that only focus on one of these two directions.

From these results, we notice in certain environments our method performs worse than the oracle baseline if no or only observation noise is present (rows 1, 5, 6, 10), but performs on the level of the oracle baseline when dynamics is added (rows 3, 4, 7, 8, 11, 12). We believe it's due to 'sliding', a default behavior in 3D simulators allowing agents to slide along obstacles rather than stopping immediately on contact. Following the findings and recommendations of [16], we disabled sliding to make our simulation results more predictive of real-world experiments. We find that one common failure mode in the absence of sliding is that agents get stuck on obstacles. In the presence of dynamics noise, the slight amount of actuation noise allows the agent to free itself from obstacles, similar to how it would in reality. Without dynamics noise, the agents continue to stay stuck.

*Sample Efficiency:* We repeat our experiments, varying the amounts of data collected from the target environment. We re-train $\mathcal{OA}$ and $\mathcal{DA}$ using 100, 250, 500, 1000, and 5000 steps of experience in the target environment, and re-evaluate performance. We compare this to directly finetuning in the target environment for varying amounts of data.

In Fig. 7, the x-axis represents the number of samples collected in the target environment. From previous experiments, we estimate 1 episode in the real-world to last on average 6 minutes, in which the robot will take approximately 70 steps to reach the goal. We use this as a conversion factor, and add an additional x-axis to show the number of hours needed for collecting the required samples from the target environment. The y-axis shows the SPL in the target environment. We see that the majority of our success comes from our first 1000 samples from the target environment, and after 5000 samples, $\pi^{\text{BDA}-5k}_{\text{Gibson}^{OA+DA}}$ is able to match the performance from $\pi^{1\text{ M}}_{\text{Gibson}^{O+D}}$. Collecting 5000 samples of data from a target environment to train our method would have taken 7 hours. In comparison, Fig. 7(b) shows that we would have to finetune the base policy for approximately 585000 steps in the target environment (836 hours to collect data from target environment) to reach the same SPL. Comparing the amount of data needed to reach the same SPL, we see that BDA reduces the amount of data needed from the target environment by 36× in Fig. 7(a), 117× in Fig. 7(b), and 32× in Fig. 7(c), for an average speed up of 61×. These results give us confidence in the importance of our approach, as we wish to limit the amount of data needed from a target environment (i.e. real-world).

## V. Related Work

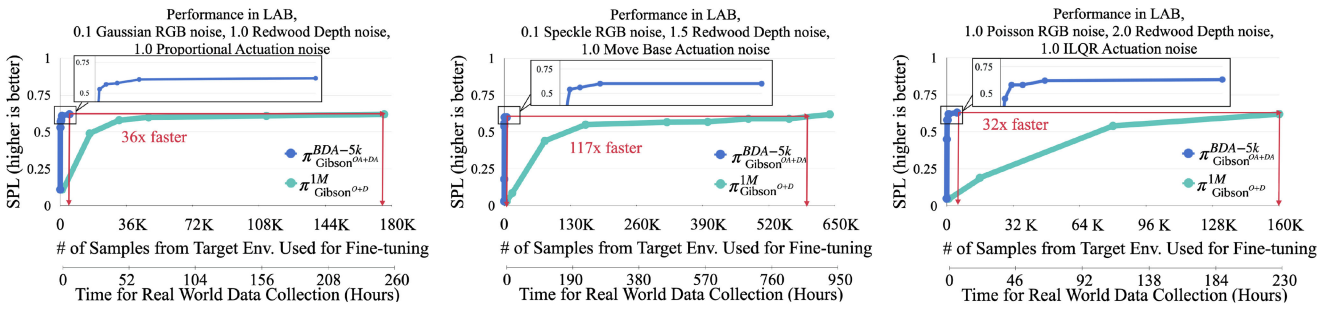
Bi-directional Domain Adaptation is related to literature on domain and dynamics randomization, domain adaptation, and residual policy learning.

*Domain and Dynamics Randomization:* Borrowing ideas from data augmentation commonly used in computer vision, domain randomization is a technique to train robust policies by exposing the agent to a wide variety of simulation environments with randomized visual properties such as lighting, texture, camera position, etc. Similarly, dynamics randomization is a process that randomizes physical properties in the simulator such as friction, mass, damping, etc. [24] applied randomization to textures to learn real indoor flight by training solely in simulation. [25] used real-world roll outs to learn a distribution of simulation dynamics parameters to randomize over. [2] randomized both physical and visual parameters to train a robotic hand to perform in hand manipulation. However, finding the right distribution to randomize parameters over is difficult, and requires expert knowledge. If the distribution chosen to randomize parameters over is too large, the task becomes much harder for the policy to learn; if the distribution is too small, then the reality gap remains large, and the policy will fail to generalize.

*Domain Adaptation:* To bridge the simulation to reality gap, many works have used domain adaptation, a technique in which data from a source domain is adapted to more closely resemble data from a target domain. Prior works have used domain adaptation techniques for adapting vision-based models to translate images from sim-to-real during training for manipulation tasks [4], [6], and real-to-sim during testing for navigation tasks [10]. Other works have focused on adapting policies for dynamic changes [5], [9]. In our work, we seek to use domain adaptation to close the gap for both the visual and the dynamics domain.

*Residual Policy Learning:* An alternative to typical transfer learning techniques is to directly improve the underlying policy itself. Instead of re-training an agent from scratch when policies perform sub-optimally, the sub-optimal policy can be used as prior knowledge in RL to speed up training. This is the main idea behind residual policy learning, in which a residual policy

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:55:33 UTC from IEEE Xplore. Restrictions apply.




(a) (b) (c)

Fig. 7. Performance of BDA compared to directly finetuning a policy in the target environment: Plots (a), (b) and (c) represent LAB environments with different noise settings we test in. On average, BDA requires 61× less data from the target environment to achieve the same SPL as finetuning directly in the target environment.

is used to augment an initial policy to correct for changes in the environment. [26], [27] demonstrated that combining residual policy learning with conventional robotic control improves the robot's ability to adapt to variations in the environment for manipulation tasks. Our method builds on this line of research by augmenting the simulator using a neural network that learns the residual error between simulation and reality.

## VI. Conclusion

We introduce Bi-directional Domain Adaptation (BDA), a method to utilize the differences between simulation and reality to accelerate learning and improve generalization of RL policies. We use domain adaptation techniques to transfer images from *real2sim* to close the visual domain gap, and learn the residual error in dynamics from *sim2real* to close the dynamics domain gap. We find that our method consistently improves performance of the initial policy $\pi$ while remaining sample efficient.

## Acknowledgment

The views and conclusions contained herein are those of the authors and should not be interpreted as necessarily representing the official policies or endorsements, either expressed or implied, of the U.S. Government, or any sponsor.

*License for dataset used:* Gibson Database of Spaces. License at http://svl.stanford.edu/gibson2/assets/GDS agreement.pdf

## References

[1] E. Wijmans *et al.*, "DD-PPO: Learning near-perfect pointgoal navigators from 2.5 billion frames," in ICLR, 2020.
[2] O. M. Andrychowicz *et al.*, "Learning dexterous in-hand manipulation," *Int. J. Robot. Res.*, vol. 39, no. 1, pp. 3–20, 2020.
[3] T. Haarnoja, S. Ha, A. Zhou, J. Tan, G. Tucker, and S. Levine, "Learning to walk via deep reinforcement learning," 2018, *arXiv:1812.11103*.
[4] K. Bousmalis *et al.*, "Using simulation and domain adaptation to improve efficiency of deep robotic grasping," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2018, pp. 4243–4250.
[5] F. Golemo, A. A. Taiga, A. Courville, and P.-Y. Oudeyer, "Sim-to-real transfer with neural-augmented robot simulation," in *Proc. Conf. Robot Learn.*, 2018, pp. 817–828.
[6] S. James *et al.*, "Sim-to-real via sim-to-sim: Data-efficient robotic grasping via randomized-to-canonical adaptation networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 12619–12629.
[7] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Sim-to-real transfer of robotic control with dynamics randomization," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2018, pp. 3803–3810.
[8] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2017, pp. 23–30.
[9] W. Yu, J. Tan, Y. Bai, E. Coumans, and S. Ha, "Learning fast adaptation with meta strategy optimization," *IEEE Robot. Automat. Lett.*, vol. 5, no. 2, pp. 2950–2957, Apr. 2020.
[10] J. Zhang *et al.*, "VR-Goggles for robots: Real-to-sim domain adaptation for visual control," *IEEE Robot. Automat. Lett.*, vol. 4, no. 2, pp. 1148–1155, Apr. 2019.
[11] F. Xia, A. R. Zamir, Z. He, A. Sax, J. Malik, and S. Savarese, "Gibson env: Real-world perception for embodied agents," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 9068–9079.
[12] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros, "Unpaired image-to-image translation using cycle-consistent adversarial networks," in *Proc. Comput. Vis., IEEE Int. Conf.*, 2017, pp. 2242–2251.
[13] R. E. Schapire, "A brief introduction to boosting," in *Proc. 16th Int. Joint Conf. Artif. Intell.*, vol. 2, 1999.
[14] P. Anderson *et al.*, "On evaluation of embodied navigation agents," 2018, *arXiv:1807.06757*.
[15] M. Savva *et al.*, "Habitat: A platform for embodied AI research," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 9339–9347.
[16] A. Kadian *et al.*, "Sim2real predictivity: Does evaluation in simulation predict real-world performance," *IEEE Robot. Automat. Lett.*, vol. 5, no. 4, pp. 6670–6677, Oct. 2020.
[17] "Locobot: An open source low cost robot," [Online]. Available: https://locobot-website.netlify.com/
[18] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: MIT Press, 2018.
[19] S. Kohlbrecher, J. Meyer, O. von Stryk, and U. Klingauf, "A flexible and scalable slam system with full 3D motion estimation," in *Proc. IEEE Int. Symp. Safety, Secur. Rescue Robot.*, Nov. 2011, pp. 155–160.
[20] S. Choi, Q.-Y. Zhou, and V. Koltun, "Robust reconstruction of indoor scenes," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2015, pp. 5556–5565.
[21] A. Murali *et al.*, "Pyrobot: An open-source robotics framework for research and benchmarking," 2019, *arXiv:1906.08236*.
[22] "Habitat challenge 2019, habitat embodied agents workshop. CVPR 2019," https://aihabitat.org/challenge/2019/.
[23] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "Gans trained by a two time-scale update rule converge to a local nash equilibrium," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 6626–6637.
[24] F. Sadeghi and S. Levine, "CAD2RL: Real single-image flight without a single real image," in *Robot.: Sci. Syst. XIII, Massachusetts Inst. Technol.*, Cambridge, Massachusetts, USA, Jul. 12-16 2017.
[25] Y. Chebotar *et al.*, "Closing the sim-to-real loop: adapting simulation randomization with real world experience," 2019, pp. 8973–8979.
[26] T. Johannink *et al.*, "Residual reinforcement learning for robot control," in *Proc. Int. Conf. Robot. Automat.*, 2019, pp. 6023–6029.
[27] T. Silver, K. R. Allen, J. B. Tenenbaum, and L. P. Kaelbling, "Residual policy learning," 2018, *arXiv:1812.06298*.

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:55:33 UTC from IEEE Xplore. Restrictions apply.